

Spring IoC Container, Beans & Annotation-Based Configuration

1. Core Idea: Classes Should Not Create Their Own Dependencies

In a well-designed application, a class should focus on its own responsibility.

For example, `OrderService` should focus on placing an order. It should not be responsible for creating the object of `PaymentService`.

```
public class OrderService {  
  
    private PaymentService paymentService = new PaymentService()  
    e();  
  
    public void placeOrder() {  
        paymentService.pay();  
        System.out.println("Order placed");  
    }  
}
```

The problem here is that `OrderService` is tightly coupled with `PaymentService`.

If tomorrow we want to replace `PaymentService` with another implementation, we will have to change the code inside `OrderService`.

A better design is:

```
public class OrderService {  
  
    private final PaymentService paymentService;
```

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}  
  
public void placeOrder() {  
    paymentService.pay();  
    System.out.println("Order placed");  
}  
}
```

Now `OrderService` does not create its dependency. It receives the dependency from outside.

This is Dependency Injection.

2. First-Principles Understanding

At the most basic level, every application needs objects.

Those objects may also need other objects.

So the real questions are:

```
Who will create these objects?  
Who will connect them together?  
Who will manage their lifecycle?
```

Without Spring, we usually do this manually.

```
main() creates objects.  
main() connects objects.  
main() behaves like a small manual container.
```

With Spring, this responsibility is shifted to the Spring IoC container.

```
Spring creates objects.  
Spring connects objects.
```

Spring manages their lifecycle.

This is the basic idea of IoC: Inversion of Control.

Instead of our code controlling object creation and dependency wiring, Spring controls it for us.

3. Basic Project Setup

To work with Spring Core using annotation-based configuration:

1. Create a Maven project.
2. Add the `spring-context` dependency.

`spring-context` gives us important container features such as:

- `ApplicationContext`
- Annotation-based configuration
- Component scanning
- Bean creation and dependency injection

4. What Is a Spring Bean?

A Spring Bean is an object managed by the Spring IoC container.

That means Spring is responsible for:

- Creating the object
- Wiring its dependencies
- Managing its lifecycle
- Giving it to us when we ask for it

Simple definition:

A Spring Bean is an object whose creation, dependency wiring, and lifecycle are managed by the Spring IoC container.

5. How Can Spring Manage Our Objects?

Spring can manage objects mainly through two configuration styles:

1. Annotation-based configuration
2. XML-based configuration

Annotation-Based Configuration

This is the modern and commonly used approach.

We use annotations such as:

- `@Component`
- `@Configuration`
- `@ComponentScan`
- `@Autowired`
- `@Bean`

XML-Based Configuration

This was widely used in older Spring applications.

Today, XML configuration is not preferred for new projects, but it is still useful to understand because many legacy projects may still contain XML-based bean configuration.

6. Reflection: Why `Student.class` Matters

When we write something like:

```
Student.class
```

we are not creating a `Student` object.

Instead, we are referring to a special object of type `Class`.

Example:

```
Class<Student> c = Student.class;
```

Here, `c` does not contain a `Student` object.

It contains metadata about the `Student` class, such as:

```
Class name    -> Student
Fields        -> name, age
Methods       -> study()
Constructors   -> Student()
Annotations   -> @Component, @Service, etc.
```

Spring uses this kind of class metadata internally.

When we pass a class like this:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

we are giving Spring metadata about the `AppConfig` class so that Spring can read configuration instructions from it.

7. Telling Spring Which Classes to Manage

Spring does not automatically manage every class in the project.

We need to tell Spring which classes are eligible to become beans.

One common way is by using `@Component`.

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment done");
    }
}
```

`@Component` tells Spring:

This class is eligible to become a Spring bean.

But just writing `@Component` is not enough.

Spring also needs to know where it should search for such classes. That is where `@ComponentScan` comes in.

8. `ApplicationContext` : The Spring IoC Container

In normal Spring applications, we commonly work with `ApplicationContext`.

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

`ApplicationContext` represents the Spring IoC container.

It is responsible for:

- Reading configuration
- Creating beans
- Resolving dependencies
- Managing bean lifecycle
- Providing beans when requested

Important point:

```
ApplicationContext context = new ApplicationContext();
```

This is not possible because `ApplicationContext` is an interface.

So we use one of its implementations.

For annotation-based configuration, we commonly use:

```
AnnotationConfigApplicationContext
```

9. What Is `AnnotationConfigApplicationContext` ?

`AnnotationConfigApplicationContext` is an implementation of `ApplicationContext`.

It starts a Spring container using Java annotation-based configuration.

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

This means:

```
Start the Spring container.  
Read instructions from AppConfig.class.  
Use annotation-based configuration.  
Create and manage beans accordingly.
```

10. What Is `AppConfig.class` ?

`AppConfig` is a configuration class.

Example:

```
@Configuration  
@ComponentScan("in.coderarmy")  
public class AppConfig {  
}
```

This class tells Spring:

```
This is a configuration class.  
Scan the package in.coderarmy.  
Find classes marked with annotations like @Component.  
Create their beans.  
Wire their dependencies.
```

So when we write:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

we are giving Spring its startup instruction file.

11. What Is `@Configuration` ?

`@Configuration` tells Spring that a class contains Spring configuration instructions.

```
@Configuration
public class AppConfig {
}
```

When Spring sees this, it understands:

```
This is not just a normal class.
This class may contain Spring setup instructions.
This class can be a source of bean definitions.
```

A configuration class may contain:

- `@ComponentScan`
- `@Bean` methods
- Other configuration-related instructions

Simple definition:

`@Configuration` marks a class as a source of bean definitions.

12. What Is `@ComponentScan` ?

When Spring starts, it needs to know where to search for classes marked with annotations like `@Component`.

```
@Configuration
@ComponentScan("com.coderarmy")
```

```
public class AppConfig {  
}
```

This tells Spring:

```
Start scanning from com.coderarmy.  
Also scan its sub-packages.  
Find classes marked with @Component, @Service, @Repository, @Controller, etc.  
Register them as beans.
```

@Component vs @ComponentScan

Annotation	Meaning
@Component	Marks a class as eligible to become a Spring bean
@ComponentScan	Tells Spring where to search for such classes

Can We Write @ComponentScan Without a Package Name?

Yes.

```
@Configuration  
@ComponentScan  
public class AppConfig {  
}
```

In this case, Spring scans the package where `AppConfig` is present and its sub-packages.

For example, if `AppConfig` is present here:

```
in.coderarmy.AppConfig
```

then Spring scans:

```
in.coderarmy
```

and its sub-packages.

13. What Does `getBean()` Mean?

After Spring creates and stores beans inside the container, we can ask the container for a bean.

```
OrderService orderService =  
    context.getBean(OrderService.class);
```

This means:

```
Spring, give me the OrderService bean from your container.
```

If Spring has already created the bean, it returns that object.

If Spring does not know about `OrderService`, we will get an error.

Common reasons:

```
OrderService is not annotated with @Component.  
The package of OrderService is not included in @ComponentScan.  
The bean is not registered using @Bean or XML.
```

Example:

```
public class Main {  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new AnnotationConfigApplicationContext(AppCon  
fig.class);  
  
        OrderService orderService =  
            context.getBean(OrderService.class);  
  
        orderService.placeOrder();  
    }  
}
```

```
}  
}
```

14. Types of Dependency Injection in Spring

Spring commonly supports three types of dependency injection:

1. Constructor injection
2. Field injection
3. Setter injection

15. Constructor Injection

In constructor injection, dependencies are provided through the constructor.

```
@Component  
public class OrderService {  
  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
  
    public void placeOrder() {  
        paymentService.pay();  
        System.out.println("Order placed");  
    }  
}
```

In modern Spring, if a bean has only one constructor, writing `@Autowired` on that constructor is optional.

Spring can automatically use that constructor for dependency injection.

This is also valid:

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

16. Why Constructor Injection Is Preferred

A constructor is called when an object is created.

So if `OrderService` needs `PaymentService`, the cleanest time to provide `PaymentService` is while creating `OrderService`.

That means the object is created in a complete and usable state.

Benefit 1: Dependency Is Mandatory

If `OrderService` cannot work without `PaymentService`, constructor injection makes that requirement clear.

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

The object cannot be created without its required dependency.

Benefit 2: We Can Use `final`

```
private final PaymentService paymentService;
```

This means once the dependency is assigned, it cannot be changed accidentally.

This makes the class safer.

Benefit 3: Easy to Test Without Spring

With constructor injection, we can manually create and test the class.

```
PaymentService paymentService = new PaymentService();
OrderService orderService = new OrderService(paymentService);

orderService.placeOrder();
```

We do not need Spring just to create the object.

This keeps our class clean, simple, and testable.

Spring helps us wire the object, but our Java class does not become tightly dependent on the Spring container.

17. Field Injection

In field injection, Spring directly injects the dependency into a field.

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This works because Spring can use reflection to set the field value.

However, field injection is generally not preferred.

Reasons:

- The dependency is hidden.
- The class cannot be easily tested without Spring.
- The field cannot be marked as `final`.
- The object can exist in an incomplete state before Spring injects the field.

Constructor injection is usually the better choice for required dependencies.

18. Setter Injection

In setter injection, Spring creates the object first and then calls a setter method to provide the dependency.

```
@Component
public class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Here, Spring follows this flow:

```
Create OrderService object using no-argument constructor.
Call setPaymentService().
Pass PaymentService into the setter.
```

Setter injection is useful when a dependency is optional or can be changed after object creation.

For mandatory dependencies, constructor injection is usually preferred.

19. What Happens Internally When Spring Starts?

When we write:

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

Spring starts the container and performs several steps.

Step 1: Spring Starts the Container

Spring creates an `ApplicationContext`.

This becomes the IoC container for our application.

Step 2: Spring Reads `AppConfig.class`

Spring looks at the class passed to it.

```
new AnnotationConfigApplicationContext(AppConfig.class);
```

It understands:

```
This class contains configuration instructions.  
I need to read annotations present on this class.
```

Step 3: Spring Processes `@ComponentScan`

If `AppConfig` contains:

```
@ComponentScan("com.coderarmy")
```

Spring understands:

```
Search inside com.coderarmy and its sub-packages.
```

Step 4: Spring Finds Component Classes

Spring searches the given package and finds classes marked with annotations such as:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

Example:

```
@Component
public class PaymentService {
}
```

```
@Component
public class OrderService {
}
```

Step 5: Spring Creates Bean Definitions

Before creating actual objects, Spring first stores information about those objects.

For example, for `PaymentService`, Spring may store metadata like:

```
Bean name      -> paymentService
Bean class     -> com.coderarmy.service.PaymentService
Scope         -> singleton
Dependencies   -> none
Creation strategy -> constructor
```

This metadata is called a `BeanDefinition`.

A `BeanDefinition` is not the actual object.

It is information about how the object should be created and managed.

20. Why Does Spring Create Bean Definitions First?

Spring is not just creating one object.

It is managing a complete application.

Before creating objects, Spring needs to know:

- Which beans exist
- What their classes are
- What their scopes are
- What dependencies they need
- How they should be created
- What lifecycle methods should be applied

That is why Spring first builds bean definitions.

Conceptually:

```
BeanDefinition list
|
├─ paymentService
└─ orderService
```

After this, Spring starts creating actual bean objects.

Step 6: Spring Creates Bean Objects

Now Spring has a list of bean definitions.

For example:

```
paymentService
orderService
```

If `PaymentService` has no dependency, Spring can create it easily.

Conceptually:

```
PaymentService paymentService = new PaymentService();
```

But this object is created internally by Spring.

Now the container has:

```
ApplicationContext
|
└─ paymentService bean
```

Step 7: Spring Creates `OrderService`

Now Spring tries to create `OrderService`.

```
@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring checks the constructor:

```
public OrderService(PaymentService paymentService)
```

Spring understands:

```
To create OrderService, I need PaymentService.
```

This is called dependency resolution.

Spring then checks:

```
Do I have a PaymentService bean in the container?
```

If yes, Spring passes that bean into the constructor.

Conceptually:

```
OrderService orderService = new OrderService(paymentService);
```

Now the container has:

```
ApplicationContext
|
├─ paymentService bean
└─ orderService bean
```

Step 8: Spring Injects Dependencies

In constructor injection, dependency injection happens at object creation time.

When Spring calls:

```
new OrderService(paymentService)
```

`PaymentService` is injected into `OrderService`.

So after creation, `OrderService` is already ready to use.

Step 9: Our Application Uses the Bean

Now we can use the bean:

```
orderService.placeOrder();
```

At this point, `OrderService` already has `PaymentService` inside it.

```
public void placeOrder() {  
    paymentService.pay();  
    System.out.println("Order placed");  
}
```

Output:

```
Payment done  
Order placed
```

21. What If No Matching Bean Exists?

Suppose `OrderService` needs `PaymentService`, but Spring does not have any `PaymentService` bean.

Then Spring cannot create `OrderService`.

We may get an error like:

```
No qualifying bean of type 'PaymentService' available
```

Common reasons:

- `PaymentService` is not annotated with `@Component`.
- Its package is not included in `@ComponentScan`.

- No `@Bean` method is creating it.
- The required dependency is not registered in Spring.

22. What If Multiple Matching Beans Exist?

Now consider this case:

```
public interface PaymentService {  
    void pay();  
}
```

```
@Component  
public class UPIPaymentService implements PaymentService {  
  
    public void pay() {  
        System.out.println("UPI payment done");  
    }  
}
```

```
@Component  
public class CardPaymentService implements PaymentService {  
  
    public void pay() {  
        System.out.println("Card payment done");  
    }  
}
```

Now `OrderService` depends on the interface:

```
@Component  
public class OrderService {  
  
    private final PaymentService paymentService;
```

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}  
}
```

Spring sees:

```
OrderService needs PaymentService.
```

But inside the container, Spring has two matching beans:

```
upiPaymentService -> UPIPaymentService -> PaymentService  
cardPaymentService -> CardPaymentService -> PaymentService
```

Both are valid implementations of `PaymentService`.

Now Spring is confused:

```
Should I inject UPIPaymentService?  
Should I inject CardPaymentService?
```

This is a bean ambiguity problem.

We can solve it using:

- `@Primary`
- `@Qualifier`

23. Using `@Primary`

`@Primary` is used when one implementation should be the default choice.

```
@Primary  
@Component  
public class UPIPaymentService implements PaymentService {
```

```

    public void pay() {
        System.out.println("UPI payment done");
    }
}

```

This tells Spring:

If multiple `PaymentService` beans are available,
prefer this one by default.

Now if `OrderService` asks for `PaymentService`, Spring will inject `UPIPaymentService`.

24. Using `@Qualifier`

`@Qualifier` is used when we want to explicitly choose a specific bean.

```

@Component
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(
        @Qualifier("cardPaymentService") PaymentService p
aymentService
    ) {
        this.paymentService = paymentService;
    }
}

```

This tells Spring:

Inject the bean named `cardPaymentService` here.

How Did Spring Get the Name `cardPaymentService` ?

By default, Spring creates the bean name from the class name.

`CardPaymentService`

becomes:

`cardPaymentService`

Similarly:

`UPIPaymentService`

becomes:

`UPIPaymentService`

In many cases, the first letter is lowercased. However, acronyms can behave differently depending on Java bean naming rules.

To avoid confusion, we can give a custom bean name.

25. Custom Bean Names with `@Component`

We can give a custom name to a component.

```
@Component("upi")
public class UPIPaymentService implements PaymentService {
}
```

```
@Component("card")
public class CardPaymentService implements PaymentService {
}
```

Now we can use:

```
public OrderService(@Qualifier("card") PaymentService payment
Service) {
    this.paymentService = paymentService;
}
```

This is clearer and easier to control.

26. `@Qualifier` with Field Injection

`@Qualifier` can also be used with field injection.

```
@Component
public class OrderService {

    @Autowired
    @Qualifier("upiPaymentService")
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

This works, but constructor injection is still usually preferred.

27. `@Qualifier` with Setter Injection

`@Qualifier` can also be used with setter injection.

```
@Component
public class OrderService {

    private PaymentService paymentService;
```

```

@Autowired
public void setPaymentService(
    @Qualifier("cardPaymentService") PaymentService p
) {
    this.paymentService = p;
}

```

28. What If Both `@Primary` and `@Qualifier` Are Used?

If both `@Primary` and `@Qualifier` are present, `@Qualifier` gets priority.

Reason:

- `@Primary` gives a default choice.
- `@Qualifier` gives a specific choice.

Specific choice wins over default choice.

29. Why Do We Need `@Bean` ?

So far, we used `@Component` to register our own classes as Spring beans.

But sometimes we cannot use `@Component`.

For example, suppose a class comes from an external library.

```

public class EmailClient {

    private final String apiKey;

    public EmailClient(String apiKey) {
        this.apiKey = apiKey;
    }

    public void sendEmail() {

```

```
        System.out.println("Email sent using API key: " + api  
Key);  
    }  
}
```

We cannot open this library class and add:

```
@Component
```

But we may still want Spring to manage its object.

In such cases, we use `@Bean`.

30. Another Case: Custom Object Creation

Sometimes object creation is not simple.

Example:

```
public class User {  
  
    private String name;  
    private String email;  
  
    public User(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

If we simply ask Spring to create this object, Spring may not know what values to pass for `name` and `email`.

So we can tell Spring exactly how to create it.

That is what `@Bean` helps us do.

31. What Exactly Does `@Bean` Mean?

`@Bean` is used on a method inside a configuration class.

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This means:

Spring, call this method.
Take the returned object.
Register it inside your container.
Manage it as a Spring bean.

For singleton beans, Spring usually creates and stores the object when the context starts.

32. Custom Bean Name with `@Bean`

By default, the bean name is the method name.

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

Bean name:

paymentService

We can also give a custom bean name.

```
@Bean("myPaymentService")
public PaymentService paymentService() {
    return new PaymentService();
}
```

Now we can fetch it by name:

```
PaymentService paymentService =
    (PaymentService) context.getBean("myPaymentService");
```

33. @Bean with Dependencies

Suppose `OrderService` needs `PaymentService`.

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }

}
```

No `@Component` is used here.

We can create both beans using `@Bean`.

```
@Configuration
public class AppConfig {
```

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}

@Bean
public OrderService orderService(PaymentService paymentService) {
    return new OrderService(paymentService);
}
}
```

Here, Spring first creates `PaymentService`.

Then it passes `PaymentService` into the `orderService()` method.

This is clean because Spring is still resolving the dependency.

Another possible style is:

```
@Bean
public OrderService orderService() {
    return new OrderService(paymentService());
}
```

In a full `@Configuration` class, Spring ensures singleton behavior for such method calls. Still, using method parameters is usually cleaner and easier to understand.

34. `@Component` vs `@Bean`

Point	<code>@Component</code>	<code>@Bean</code>
Where used?	On a class	On a method
Style	Automatic detection	Manual registration
Best for	Our own classes	External library classes or custom object creation
Requires component scanning?	Yes	No, but configuration class must be loaded

Point	@Component	@Bean
Bean name	Class name based by default	Method name by default

Simple comparison:

```
@Component = Spring finds the class automatically.
@Bean       = We manually tell Spring how to create the object.
```

35. Important Note: Avoid Creating the Same Bean Twice

Do not register the same object through both `@Component` and `@Bean` unless you intentionally want multiple beans.

Example:

```
@Component
public class PaymentService {
}
```

and also:

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

This can create confusion.

Possible outcomes:

- Spring may treat them as two different beans if their names are different.
- Dependency injection by type may become ambiguous.
- If the names are same, bean overriding rules may apply depending on configuration.

So the clean rule is:

Register a bean using either `@Component` or `@Bean`, not both, unless you have a clear reason.

36. Dependency Resolution with `@Bean`

If multiple beans of the same type exist, the same rules apply.

We can use `@Primary`.

```
@Bean
@Primary
public PaymentService upiPaymentService() {
    return new UPIPaymentService();
}
```

We can use `@Qualifier`.

```
@Bean
public OrderService orderService(
    @Qualifier("cardPaymentService") PaymentService paymentService
) {
    return new OrderService(paymentService);
}
```

This tells Spring exactly which bean to inject.

37. Why Not Put Everything in `main()` ?

A common question is:

Why do we need `AppConfig` ? Why not create everything in `main()` ?

Because `main()` should only start the application.

If we put all object creation and wiring logic inside `main()`, then `main()` again becomes responsible for configuration and object management.

That brings us back to manual object creation.

Instead, we keep `main()` simple:

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppCon
fig.class);

        OrderService orderService =
            context.getBean(OrderService.class);

        orderService.placeOrder();
    }
}
```

Now object creation, dependency wiring, and lifecycle management are handled by Spring.

38. BeanFactory vs ApplicationContext

Internally, Spring has something called `BeanFactory`.

`BeanFactory` is the basic container interface.

`ApplicationContext` builds on top of it and provides more complete features for real applications.

In normal Spring applications, we commonly use `ApplicationContext`.

Simple understanding:

```
BeanFactory      -> Basic container
ApplicationContext -> Advanced container used commonly in applications
```

`ApplicationContext` gives us features like:

- Bean creation and dependency injection
- Lifecycle management
- Event publishing
- Internationalization support
- Integration with other Spring features

39. Final Flow Summary

When we start Spring using:

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

Spring performs this flow:

1. Start the IoC container.
2. Read AppConfig.class.
3. Process @Configuration.
4. Process @ComponentScan.
5. Scan the given package and sub-packages.
6. Find classes marked with @Component and related annotations.
7. Create BeanDefinitions.
8. Resolve dependencies.
9. Create bean objects.
10. Inject dependencies.
11. Store beans inside the container.
12. Return beans when getBean() is called.

40. Key Takeaways

- A Spring Bean is an object managed by the Spring IoC container.
- `ApplicationContext` represents the Spring IoC container in normal applications.

- `AnnotationConfigApplicationContext` starts a container using annotation-based configuration.
- `@Configuration` marks a class as a source of Spring configuration.
- `@ComponentScan` tells Spring where to search for components.
- `@Component` marks a class as eligible to become a bean.
- `getBean()` asks Spring for an object from the container.
- Constructor injection is generally preferred for mandatory dependencies.
- Field injection works, but it is not recommended for clean design.
- Setter injection is useful for optional or changeable dependencies.
- Spring first creates `BeanDefinition` metadata, then creates actual bean objects.
- If no matching bean exists, Spring throws an error.
- If multiple matching beans exist, Spring needs help through `@Primary` or `@Qualifier`.
- `@Bean` is useful when we want to manually tell Spring how to create an object.
- `@Component` is class-level and automatic.
- `@Bean` is method-level and manual.
- Avoid registering the same bean through both `@Component` and `@Bean` unless you intentionally want multiple beans.
- `main()` should start the application, not manage all object creation.
- `ApplicationContext` is commonly used because it provides complete container features.

41. One-Line Revision

Spring IoC Container creates objects, wires dependencies, manages lifecycle, and gives us ready-to-use beans when required.